

COMM114: Generative AI - VAE (cont.)

Guoqiang Zhang

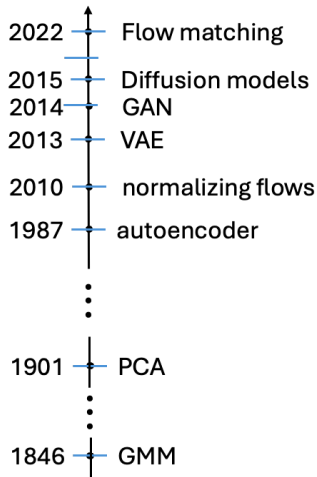
September 2025

Table of Contents

- How ELBO is derived?
- Sampling
- Conditional VAE

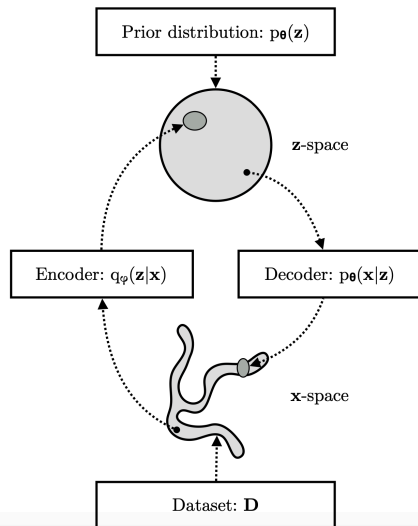
VAE

Variational autoencoder (VAE) was invented in 2013 by Diederik P. Kingma and Max Welling.



Slide from Last Lecture

- A VAE learns stochastic mappings between an observed x -space and a latent z space.
- The generative model learns a joint distribution $p_{\theta}(x, z) = p_{\theta}(z)p_{\theta}(x|z)$.
- $p_{\theta}(z)$ is the prior distribution of the latent random variable z .
- The conditional distribution $p_{\theta}(x|z)$ is learned by the stochastic decoder.
- The stochastic encoder learns an approximation $q_{\varphi}(z|x)$ of the true but intractable posterior $p_{\theta}(z|x)$ of the generative model.



Remark: $q_{\varphi}(z|x)$ is referred to as the approximate posterior.

Slide from Last Lecture

The stochastic encoder (parameterized by φ) and the stochastic decoder (parameterized by θ) in a VAE are trained by maximizing a so-called **evidence lower bound (ELBO)** function:

$$\begin{aligned} & \max_{\theta, \varphi} \mathcal{L}(\theta, \varphi) \\ &= \max_{\theta, \varphi} \frac{1}{n} \sum_{i=1}^n [\mathcal{L}(\theta, \varphi; \mathbf{x}_i)] \\ &= \max_{\theta, \varphi} \frac{1}{n} \sum_{i=1}^n \left[-D_{KL}(q_{\varphi}(\mathbf{z}|\mathbf{x}_i)||p_{\theta}(\mathbf{z})) + \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x}_i)} [\log p_{\theta}(\mathbf{x}_i|\mathbf{z})] \right] \end{aligned}$$

Question: How is the above ELBO obtained in the very beginning?

Derivation of ELBO

Assumption

- Suppose the generative model of the VAE is represented by the decomposable joint probability distribution $p_{\theta}(z, \mathbf{x}) = p_{\theta}(\mathbf{x}|z)p_{\theta}(z)$.
 - $p_{\theta}(z)$ denotes the prior distribution of the latent variable z .
 - The conditional distribution $p_{\theta}(\mathbf{x}|z)$ is captured by the stochastic decoder in the VAE.
-
- Input: data samples $\{\mathbf{x}_i\}_{i=1}^n$
 - Objective: We would like to maximize the log-likelihood $\frac{1}{n} \sum_{i=1}^n \log p_{\theta}(\mathbf{x}_i)$

$$\arg \max_{\theta} \frac{1}{n} \sum_{i=1}^n \log p_{\theta}(\mathbf{x}_i). \quad (1)$$

Derivation of ELBO (cont.)

Each $\log p_{\theta}(\mathbf{x}_i)$ in (1) can be lower-bounded by

$$\log p_{\theta}(\mathbf{x}_i) = \log p_{\theta}(\mathbf{x}_i) \int q_{\varphi}(\mathbf{z}|\mathbf{x}_i) d\mathbf{z}$$

Multiply by 1

$$\stackrel{(a)}{=} \int \log p_{\theta}(\mathbf{x}_i) q_{\varphi}(\mathbf{z}|\mathbf{x}_i) d\mathbf{z}$$

Bring inside the integral

$$\stackrel{(b)}{=} \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x}_i)} [\log p_{\theta}(\mathbf{x}_i)]$$

Definition of expectation

$$\stackrel{(c)}{=} \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x}_i)} \left[\log \frac{p_{\theta}(\mathbf{x}_i, \mathbf{z})}{p_{\theta}(\mathbf{z}|\mathbf{x}_i)} \right]$$

Applying $p_{\theta}(\mathbf{x}_i) = \frac{p_{\theta}(\mathbf{x}_i, \mathbf{z})}{p_{\theta}(\mathbf{z}|\mathbf{x}_i)}$

$$\stackrel{(d)}{=} \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x}_i)} \left[\log \frac{p_{\theta}(\mathbf{x}_i, \mathbf{z}) q_{\varphi}(\mathbf{z}|\mathbf{x}_i)}{p_{\theta}(\mathbf{z}|\mathbf{x}_i) q_{\varphi}(\mathbf{z}|\mathbf{x}_i)} \right]$$

Multiplying by 1

$$\stackrel{(e)}{=} \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x}_i)} \left[\log \frac{p_{\theta}(\mathbf{x}_i, \mathbf{z})}{q_{\varphi}(\mathbf{z}|\mathbf{x}_i)} \right] + \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x}_i)} \left[\log \frac{q_{\varphi}(\mathbf{z}|\mathbf{x}_i)}{p_{\theta}(\mathbf{z}|\mathbf{x}_i)} \right]$$

Split the expectation

$$\stackrel{(f)}{=} \underbrace{\mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x}_i)} \left[\log \frac{p_{\theta}(\mathbf{x}_i, \mathbf{z})}{q_{\varphi}(\mathbf{z}|\mathbf{x}_i)} \right]}_{\text{ELBO}} + \underbrace{D_{KL}(q_{\varphi}(\mathbf{z}|\mathbf{x}_i) || p_{\theta}(\mathbf{z}|\mathbf{x}_i))}_{\geq 0}$$

Definition of KL divergence

Derivation of ELBO (cont.)

- By ignoring the KL divergence term, the log likelihood $\log p_{\theta}(\mathbf{x}_i)$ can be lower bounded by the evidence lower bound (ELBO):

$$\log p_{\theta}(\mathbf{x}_i) \geq \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x}_i)} \left[\log \frac{p_{\theta}(\mathbf{x}_i, \mathbf{z})}{q_{\varphi}(\mathbf{z}|\mathbf{x}_i)} \right].$$

- Instead of maximizing the log-likelihood $\frac{1}{n} \sum_{i=1}^n \log p_{\theta}(\mathbf{x}_i)$ directly, we consider maximizing the ELBO instead:

$$\arg \max_{\theta, \varphi} \frac{1}{n} \sum_{i=1}^n \mathbb{E}_{q_{\varphi}(\mathbf{z}|\mathbf{x}_i)} \left[\log \frac{p_{\theta}(\mathbf{x}_i, \mathbf{z})}{q_{\varphi}(\mathbf{z}|\mathbf{x}_i)} \right]$$

Equivalent Representation of ELBO

$$\arg \max_{\theta, \varphi} \frac{1}{n} \sum_{i=1}^n \mathbb{E}_{q_{\varphi}(z|x_i)} \left[\log \frac{p_{\theta}(x_i, z)}{q_{\varphi}(z|x_i)} \right]$$

$$\stackrel{(a)}{=} \arg \max_{\theta, \varphi} \frac{1}{n} \sum_{i=1}^n \mathbb{E}_{q_{\varphi}(z|x_i)} \left[\log \frac{p_{\theta}(x_i|z)p_{\theta}(z)}{q_{\varphi}(z|x_i)} \right]$$

Chain rule of probability

$$\stackrel{(b)}{=} \arg \max_{\theta, \varphi} \frac{1}{n} \sum_{i=1}^n \left[\mathbb{E}_{q_{\varphi}(z|x_i)} [\log p_{\theta}(x_i|z)] + \mathbb{E}_{q_{\varphi}(z|x_i)} \left[\log \frac{p_{\theta}(z)}{q_{\varphi}(z|x_i)} \right] \right]$$

Split the expectation

$$\stackrel{(c)}{=} \arg \max_{\theta, \varphi} \frac{1}{n} \sum_{i=1}^n \left[\underbrace{\mathbb{E}_{q_{\varphi}(z|x_i)} [\log p_{\theta}(x_i|z)]}_{\text{1st term}} - \underbrace{D_{KL}(q_{\varphi}(z|x_i) || p_{\theta}(z))}_{\text{2nd term}} \right]$$

Defin. of KL divergence,

which is exactly the same expression for ELBO as in our last lecture.

- Maximizing the 1st term in order to minimize the reconstruction error between the input and the associated output from the decoder.
- Minimizing the 2nd term to minimize the distance between the learned approximate posterior $p_{\varphi}(z|x_i)$ and the prior distribution $p_{\theta}(z)$ of the latent variable z .

Recall from Last Lecture

- Simplifying the training process by setting

$$\begin{aligned}p_{\theta}(\mathbf{z}) &= \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I}) \\p_{\varphi}(\mathbf{z}|\mathbf{x}_i) &= \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}^{(i)}, \boldsymbol{\sigma}^{2,(i)}\mathbf{I}) \\p_{\theta}(\mathbf{x}_i|\mathbf{z}_{i,l}) &= \mathcal{N}(\mathbf{x}_i; f_{\theta}(\mathbf{z}_{i,l}), \mathbf{I}),\end{aligned}$$

where $\mathbf{z}_{i,l} = \boldsymbol{\mu}^{(i)} + \boldsymbol{\sigma}^{(i)} \odot \boldsymbol{\epsilon}_l$, where $\boldsymbol{\epsilon}_l \sim \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$, where $l = 1, \dots, L$.

- With the above expression, the ELBO can be further approximated to be

$$\arg \min_{\boldsymbol{\theta}, \boldsymbol{\varphi}} \frac{1}{n} \sum_{i=1}^n \left[\underbrace{\frac{1}{2} \sum_{j=1}^k \left(1 + \log(\boldsymbol{\sigma}^{(i)}[j])^2 - (\boldsymbol{\mu}^{(i)}[j])^2 - (\boldsymbol{\sigma}^{(i)}[j])^2 \right)}_{\text{1st term}} + \frac{1}{L} \sum_{l=1}^L \underbrace{(\mathbf{x}_i - f_{\theta}(\mathbf{z}_{i,l}))}_{\text{2nd term}} \right]$$

Remark: The encoder and decoder of a VAE can then be trained by applying SGD to minimize the above ELBO function in terms of $\boldsymbol{\varphi}$ and $\boldsymbol{\theta}$.

Training A VAE with BCE loss for 2nd term

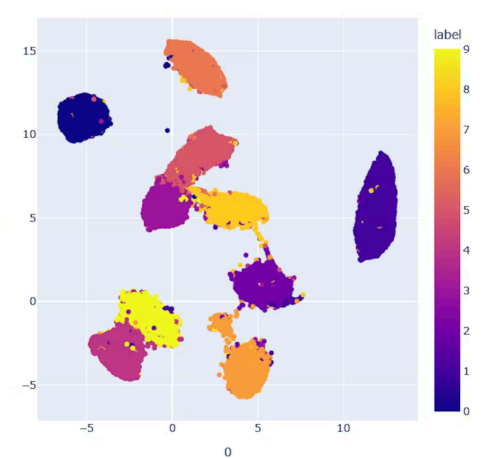
- Assume each component $\mathbf{x}_i[j] \in [0, 1], j = 1, \dots, d$.
- Treat $\mathbf{x}_i[j]$ as a probability.
- With the above assumption, the ELBO can be alternatively approximated as

$$\arg \min_{\boldsymbol{\theta}, \boldsymbol{\varphi}} \frac{1}{n} \sum_{i=1}^n \left[\underbrace{\frac{1}{2} \sum_{j=1}^k \left(1 + \log(\boldsymbol{\sigma}^{(i)}[j])^2 - (\boldsymbol{\mu}^{(i)}[j])^2 - (\boldsymbol{\sigma}^{(i)}[j])^2 \right)}_{\text{1st term}} + \frac{1}{L} \sum_{l=1}^L \sum_{j=1}^d \underbrace{-\mathbf{x}_i[j] \log f_{\boldsymbol{\theta}}(\mathbf{z}_{i,l})[j] - (1 - \mathbf{x}_i[j]) \log(1 - f_{\boldsymbol{\theta}}(\mathbf{z}_{i,l}))}_{\text{2nd term}} \right],$$

Remark: The encoder and decoder of a VAE can then be trained by applying SGD to minimize the above ELBO function in terms of $\boldsymbol{\varphi}$ and $\boldsymbol{\theta}$.

Visualization of latent space in a VAE

- The VAE is trained over MNIST dataset, which has 10 classes of handwritten digitals.
- The outputs $\{\mu^{(i)}, \sigma^{(i)}\}_{i=1}^n$ from the encoder has roughly 10 clusters, one cluster for each class.
- $(\mu^{(i)}, \sigma^{(i)})$ is the output of sample x_i .

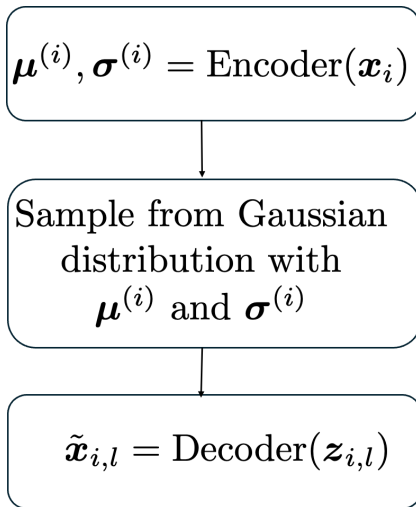


Sampling

Suppose a VAE is trained sufficiently well, we consider generating new samples from the pre-trained VAE.

- **Posterior sampling**: sampling from the posterior distribution $p_{\varphi}(z|x_i) = \mathcal{N}(z; \mu^{(i)}, \sigma^{2,(i)} I)$ given a provided input x_i , and then pass it to the decoder.
- **Prior sampling**: sampling from the prior distribution $p_{\theta}(z) = \mathcal{N}(z; \mathbf{0}, I)$, and then pass it to the decoder.
- The output for posterior sampling is a variant of the input data sample.

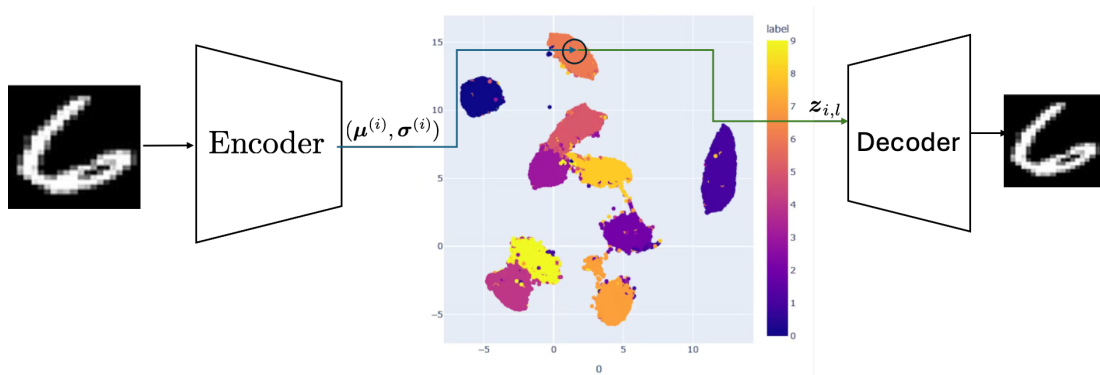
Posterior Sampling



- The output for posterior sampling is a variant of the input data sample.
- $z_{i,l} = \mu^{(i)} + \sigma^{(i)} \epsilon_l$, where $\epsilon_l \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

Posterior Sampling (cont.)

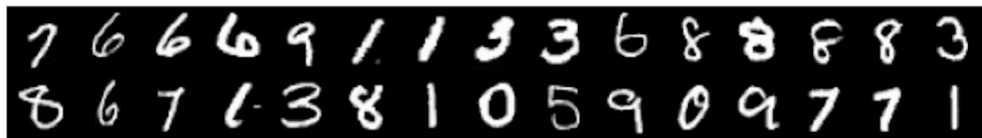
$$\mathbf{z}_{i,l} = \boldsymbol{\mu}^{(i)} + \boldsymbol{\sigma}^{(i)} \boldsymbol{\epsilon}_l$$



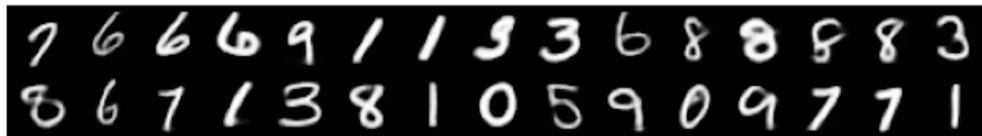
- The output for posterior sampling is a variant of the input data sample.
- $\boldsymbol{\epsilon}_l$ in the figure is a Gaussian noise drawn from $\mathcal{N}(\mathbf{0}, \mathbf{I})$.

Posterior Sampling (cont.)

Input samples

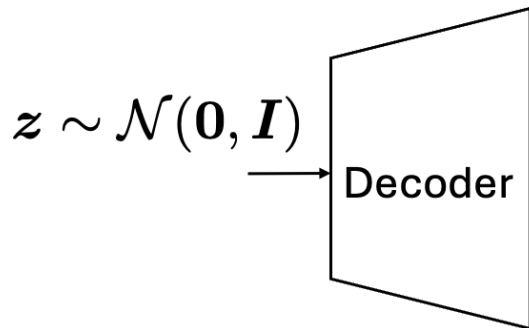


Generated posterior samples



Prior Sampling

- First generate Gaussian noises from the prior distribution and then feed the generated noises to the decoder.



Prior Sampling (cont.)

```
def prior_sampling(model, z_dim=32, n_samples = 25):  
    model.eval()  
    input_sample=torch.randn(n_samples, z_dim).to(device)  
    with torch.no_grad():  
        sampled_images = model.decoder(input_sample)  
        show_images_grid(sampled_images, title=f'generated prior samples')  
  
prior_sampling(vae, z_dim, n_samples=40)
```

Prior Sampling (cont.)

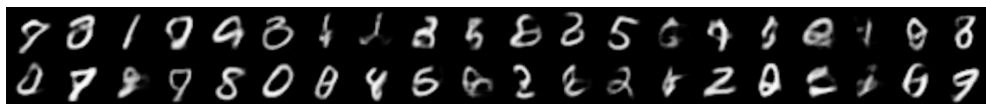


Figure: Generated prior samples

Remark: The generated samples do not have the same quality as the posterior sampling.

Conditional VAE

- A conditional VAE takes the labels of the data samples as additional input to the encoder and decoder for performance improvement.

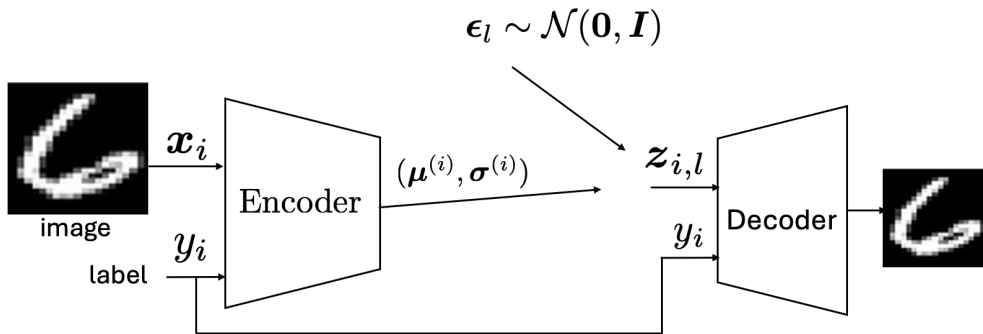


Figure: Diagram for training and sampling from a conditional VAE

Conditional VAE (cont.)

- Probability setup in a conditional VAE

$$p_{\theta}(z) = \mathcal{N}(z; \mathbf{0}, \mathbf{I})$$

$$p_{\varphi}(z|x_i, y_i) = \mathcal{N}(z; \boldsymbol{\mu}^{(i)}, \boldsymbol{\sigma}^{2,(i)} \mathbf{I}) \quad [y_i \text{ as input to encoder}]$$

$$p_{\theta}(x_i|z_{i,l}, y_i) = \mathcal{N}(x_i; f_{\theta}(z_{i,l}, y_i), \mathbf{I}), \quad [y_i \text{ as input to decoder}]$$

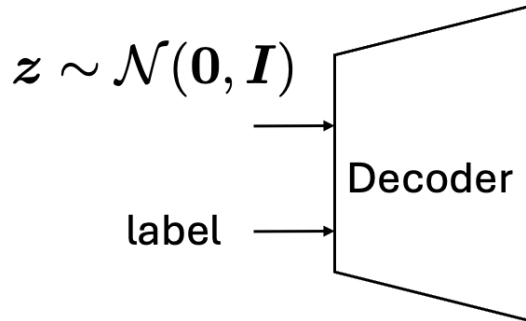
where $z_{i,l} = \boldsymbol{\mu}^{(i)} + \boldsymbol{\sigma}^{(i)} \odot \boldsymbol{\epsilon}_l$, where $\boldsymbol{\epsilon}_l \sim \mathcal{N}(z; \mathbf{0}, \mathbf{I})$, where $l = 1, \dots, L$.

- With the above expression, the ELBO can be further approximated to be

$$\arg \min_{\theta, \varphi} \frac{1}{n} \sum_{i=1}^n \left[\underbrace{\frac{1}{2} \sum_{j=1}^k \left(1 + \log(\boldsymbol{\sigma}^{(i)}[j])^2 - (\boldsymbol{\mu}^{(i)}[j])^2 - (\boldsymbol{\sigma}^{(i)}[j])^2 \right)}_{\text{1st term}} + \frac{1}{L} \sum_{l=1}^L \underbrace{(\mathbf{x}_i - f_{\theta}(z_{i,l}, y_i))^2}_{\text{2nd term}} \right],$$

Prior Sampling for a conditional VAE

- When performing prior sampling, taking the label as an additional input to the decoder.



On Taking Labels as Inputs to a DNN Model

- Input: $\{\mathbf{x}_i, y_i\}_{i=1}^n$
- Converting each y_i into a one-hot vector $\mathbf{v}_i \in \mathbb{R}^d$.
 - Example: suppose the discrete set $\Omega = \{\text{"red"}, \text{"blue"}, \dots, \text{"green"}\}$ has d elements.
 - $\mathbf{v}_i = \text{one-hot}(y_i = \text{"red"}) = [1, 0, \dots, 0]$
 - $\mathbf{v}_i = \text{one-hot}(y_i = \text{"blue"}) = [0, 1, \dots, 0]$.
 - $\mathbf{v}_i = \text{one-hot}(y_i = \text{"green"}) = [0, 0, \dots, 1]$.
- Feed each pair $(\mathbf{x}_i, \mathbf{v}_i)$ to a DNN model (e.g., the encoder and/or decoder of a VAE).

References

- Diederik P. Kingma and Max Welling, "An Introduction to Variational Autoencoders", <https://arxiv.org/abs/1906.02691>, 2019.
- Youtube video by Umar Jamil
- How to Sample From Latent Space With Variational Autoencoder by Dmitrii Matveichev

Questions?